Firmware Function Reference (TM4C1294 – integr_v03)

This document is a function-by-function reference for the current firmware in this workspace. It focuses on the UART console/diagnostics path and the modules that were actively changed during the recent UART UX + ESP32-feature recovery work.

UART Roles (High-Level)

- **UART3 (USER, 115200)**: interactive console. RX is ISR-driven (`USERUARTIntHandler`) with echo + in-ISR line editing.
- **UART0 (ICDI, 9600)**: diagnostics/status output. Runtime diagnostics are gated by `DEBUG ON/OFF`.

Session Boundary (DTR on PQ1)

- DTR is read from **PQ1** (`DTR_PORT/DTR_PIN`) and is **polled in the main loop**.
- Because DTR is polled (not interrupt-driven), the session loop uses periodic `SysCtlDelay(...)` to ensure disconnects are detected promptly (no "press ENTER to notice disconnect" behavior).

---

main.c

Global state

- `g_ui32SysClock`: system clock (Hz), set by `setup_system_clock()`.
- PWM state:
  - `g_pwmPeriod`: PWM period (ticks).
  - `g_pwmPulse`: PWM pulse width (ticks).
- UART3 RX/command state (ISR-owned, main loop consumes):
  - `user_rx_buf[]`: UART3 line accumulator.
  - `user_rx_len`: current bytes accumulated.
  - `user_cmd_ready`: set by ISR when a non-empty line is completed.
- GOTCHA hidden trigger state (UART3 ISR):
  - `g_uart3_p_run`: count of consecutive `P` keystrokes.
  - `g_uart3_gotcha_pending`: set when 5 consecutive `P` are typed.
- UART0 diagnostics gating:
  - `g_debug_enabled`: default `false`.

`void debug_set_enabled(bool enabled)`

Enables/disables UART0 diagnostic output at runtime.

- Called from the `DEBUG ON/OFF` command via commands.c.

`bool debug_is_enabled(void)`

Returns the current diagnostic gating state.

`void pwm_set_percent(uint32_t percent)`

Public wrapper used by the command layer.

- Delegates to `set_pwm_percent(percent)`.

`void pwm_set_enabled(bool enabled)`

Enables/disables PWM output on PF2.

- `enabled=true`: restores PF2 mux to `M0PWM2` and enables PWM output.
- `enabled=false`: disables PWM output and reconfigures PF2 as GPIO output low.

This exists primarily to support `PSYN OFF` for scope/debug, so the tach input can be observed without PWM coupling.

`bool pwm_is_enabled(void)`

Returns the current PWM output enabled state.

`static void set_pwm_percent(uint32_t percent)`

Sets PWM duty cycle without disabling/re-enabling the generator.

- Bounds/clamps: `percent` is clamped to 0..100.
- Ensures pulse width remains in `1..(period-1)`.
- Calls:
  - `PWMPulseWidthSet(PWM0_BASE, PWM_OUT_2, pulse)`

`static void setup_system_clock(void)`

Configures system clock to 120 MHz via PLL.

- Sets `g_ui32SysClock`.

`static void setup_pwm_pf2(void)`

Configures PWM output on **PF2 / M0PWM2**.

- Enables PWM0 + GPIOF.
- Uses `PWM_SYSCLK_DIV_1`.
- Computes and stores `g_pwmPeriod`.

`static void setup_uarts(void)`

Configures UART0 and UART3, plus supporting GPIO.

- UART0: PA0/PA1, 9600 8N1.
- UART3: PJ0/PJ1, 115200 8N1.
- PF4 configured as GPIO output (used as RX activity LED and GOTCHA blink).
- PQ1 configured as input with WPU (DTR detect).

- Enables interrupts:
  - `INT_UART0` → `ICDIUARTIntHandler()`
  - `INT_UART3` → `USERUARTIntHandler()`

`void ICDIUARTIntHandler(void)`

UART0 ISR.

- Echoes RX bytes back to UART0.
- Briefly pulses PN0 for visibility.

`void USERUARTIntHandler(void)`

UART3 ISR: echo + line accumulation + basic line editing.

Behavior summary:

- **Backspace/Delete** (`\b` or `0x7F`):
  - Deletes one buffered character if `user_rx_len > 0`.
  - Emits `"\b \b"` erase sequence.
  - If buffer empty, emits bell (`\a`) to prevent erasing past the prompt boundary.
- **ENTER** (`\r` or `\n`):
  - If buffer non-empty: emits `\r\n`, NUL-terminates buffer, sets `user_cmd_ready=true`.
  - If buffer empty: does nothing (no extra newline/prompt spam).
- **Uppercase-as-you-type**: converts `a..z` to `A..Z` before echo and buffering.
- **Hidden GOTCHA**:
  - Counts consecutive `P` keystrokes.
  - On 5 consecutive `P`, sets `g_uart3_gotcha_pending=true` and resets the counter.
  - Not a command; does not require ENTER; not listed in HELP.
- **Overflow**:
  - Resets buffer and prints `ERROR: line too long` + prompt.

`static void user_uart3_consume_pending_input(void)`

Consumes pending UART3 RX bytes while UART3 interrupts are disabled.

Why it exists:

- When hosts send CRLF, the ISR may complete the line on `\r` and then later receive/echo the trailing `\n`.
- If the main loop prints the next prompt while UART3 interrupts are disabled, the delayed `\n` can arrive after the prompt and move the cursor, creating the "extra ENTER required" UX symptom.

What it does:

- While UART3 has bytes available:
  - Swallows extra `\r/\n` tails.
  - Echoes other bytes and appends them to the current buffer (if a command isn't already pending).

`static void flash_pf4_gotcha(uint32_t flashes)`

Flashes PF4 LED `flashes` times.

- Used when GOTCHA triggers.
- Delay is derived from `g_ui32SysClock` (currently ~75ms on/off).

`void example_dynamic_cmd_copy_and_process(const volatile char *user_rx_buf, uint32_t len)`

UART0-only diagnostics helper.

- Runs only when `debug_is_enabled()`.
- Uses `diag_uart` helpers to dump internal state and stress some malloc/formatting paths.

`int main(void)`

Main firmware entry.

Execution overview:

1. Clock + PWM + UART setup.
2. Outer loop waits for DTR session.
3. On session begin:
   - UART0 prints "SESSION WAS INITIATED".
   - UART3 prints rainbow banner + welcome + prompt via `ui_uart3_session_begin()`.
4. Session loop:
   - Polls DTR.
   - If `g_uart3_gotcha_pending` is set:
     - Prints UART0 message immediately.
     - Flashes PF4.
   - If `user_cmd_ready`:
     - Disables UART3 IRQ.
     - Copies buffered line to local storage.
     - Clears ISR-owned state.
     - Calls `user_uart3_consume_pending_input()`.
     - Re-enables UART3 IRQ.
     - Dispatches the command via `commands_process_line(cmd_local)`.
     - If `DEBUG` enabled: prints additional UART0 diagnostics.
   - Uses a short `SysCtlDelay(...)` to ensure DTR polling stays responsive.
5. On disconnect:
   - UART0 prints "SESSION WAS DISCONNECTED" immediately (no user keystrokes required).

---

commands.c / commands.h

`commands_process_line()` implements the UART3 command dispatcher.

Design notes:

- Avoids `snprintf`/newlib printf-family in the command response path.
- Uses simple parsing (`strtok_r`) and a small decimal formatter.

`void commands_process_line(const char *line)`

Parses and executes one complete command line.

- Trims leading whitespace.
- Uppercases command token.
- Supported commands (authoritative list: mirrors the `HELP` output in `commands.c`):
  - `PSYN n` — set PWM duty (n=5..96).
  - `PSYN ON` — enable PWM on PF2.
  - `PSYN OFF` — disable PWM and force PF2 low.
  - `PHASE1` / `PHASE 1` — preset: PWM 24.9kHz@46% + TACHSYN 168Hz@50% on PM3.
  - `PHASE2` / `PHASE 2` — preset: PWM 24.9kHz@54% + TACHSYN 235Hz@50% on PM3.
  - `PHASE1L` / `PHASE 1L` — preset: PWM 24.9kHz@15% + TACHSYN 168Hz@50% on PM3.
  - `PHASE2L` / `PHASE 2L` — preset: PWM 24.9kHz@21% + TACHSYN 235Hz@50% on PM3.
  - `TSYN BOOT BEGIN` — start boot-profile TACHSYN on PM3.
  - `TSYN BOOT END` — stop boot-profile.
  - `TSYN COPY BEGIN` — mirror TACHIN -> TACHOUT (PF1 -> PM3).
  - `TSYN COPY END` — stop mirroring and restore the persisted `TACH DEFAULT`.
  - `TSYN STATUS` — show tach generator status.
  - `TACHIN ON` — start printing RPM on UART0 every 0.5s.
  - `TACHIN OFF` — stop printing RPM on UART0.
  - `BOTHIN ON` — start printing combined PWM duty + tach RPM on UART0 every 1s.
  - `BOTHIN OFF` — stop printing the combined line.
  - `PWMIN ON` — start printing PWM-in on UART0 every 1s (frequency + duty only).
  - `PWMIN OFF` — stop printing PWM-in on UART0.
  - `PWMINDBG ON` — enable PWMIN verbose diagnostics on UART0.
  - `PWMINDBG OFF` — disable PWMIN verbose diagnostics.
  - `PWMINDBG DUMP` — print a one-time PWMIN diagnostic dump on UART0.
  - `TACH LOOPBACK BEGIN` — self-test (jumper PM3->PF1).
  - `TACH LOOPBACK END` — stop self-test and restore `TACH DEFAULT`.
  - `TACH DEFAULT <1|2|1L|2L|BOOT|COPY>` — persist boot default.
  - `TACH DEFAULT CURRENT` — show persisted default.
  - `CLEAR` — clear terminal screen (ANSI).

  - `HELP` — print help.
  - `EXIT` — close UART3 session.
  - `DEBUG ON` — enable UART0 diagnostics.
  - `DEBUG OFF` — disable UART0 diagnostics.

Deprecated compatibility aliases:

- `TSYN ON` / `TSYN OFF` — legacy aliases retained for compatibility (maps to `TSYN BOOT BEGIN/END`).

`void pwm_set_percent(uint32_t percent)` (declared in commands.h)

Platform-provided PWM setter (implemented in main.c).

`void debug_set_enabled(bool enabled)` / `bool debug_is_enabled(void)` (declared in commands.h)

Platform-provided debug gating API (implemented in main.c).

---

timebase.c / timebase.h

Minimal SysTick-based timebase used by the tach sensing path.

`void timebase_init(uint32_t sysClockHz)`

Initializes SysTick to generate a 1ms interrupt and establishes the reference clock for cycle-based delta timing.

- Configures a 1ms tick using `SysTickPeriodSet(sysClockHz / 1000)`.
- Enables SysTick interrupt and SysTick counter.
- Stores:
  - `g_sysclk_hz` (for later conversion and debug)
  - `g_systick_reload` (cycles per millisecond)

Dependency note:

- The SysTick vector in TM4C1294XL_startup.c must point to `SysTickIntHandler()`.

`uint32_t timebase_millis(void)`

Returns a monotonically increasing millisecond tick counter.

- Implemented as an ISR-incremented counter (`g_ms_ticks`).
- Read uses a short global interrupt disable/enable to snapshot consistently.

`uint32_t timebase_cycles32(void)`

Returns a 32-bit "cycle-ish" counter derived from SysTick.

- Computes:
  - `cycles = ms * reload + (reload - SysTickValueGet())`
- Samples `g_ms_ticks` twice to avoid race at the millisecond boundary.
- Intended for **short delta measurements**; wraps naturally at 32 bits.

`uint32_t timebase_sysclk_hz(void)`

Returns the system clock rate passed into `timebase_init()`.

---

tach.c / tach.h

Interrupt-driven TACH (fan tachometer) input sensing with a simple glitch reject filter, plus optional periodic UART0 reporting.

This implementation is intentionally "debug-first": it is good enough to diagnose coupling/noise patterns and compare strategies against the ESP32 reference (`fan_master_s2_final.ino`), but it is not yet presented as a final/production tach algorithm.

Wiring and default pinning

Default configuration (compile-time override via macros in tach.h):

- TACH input: **PF1 / GPIOF1**
- Electrical assumption: open-collector/open-drain tach output.
- Uses internal weak pull-up (`GPIO_PIN_TYPE_STD_WPU`, to 3.3V).

Safety note:

- Treat the tach input as **3.3V only** unless you have verified the pull-up rail.
- If the source pull-up might be +5V, use an interface stage (transistor/MOSFET or divider + clamp) instead of wiring directly.

Pin Separation (Jan 2026)

- **TACHSYN output (faked tach to PSU)**: **PM3** (driven by `PHASE*` and `TSYN`)
- **TACH input (sense real fan tach)**: **PF1** (used by `TACHIN ON/OFF` reporting)

This separation avoids a hardware-risky conflict where one firmware mode drives a pin while another mode reconfigures that same pin as an input. ### `void tach_init(void)`

	Initializes the GPIO and interrupt configuration for tach capture.

	- Configures pad: - input, 2mA drive (irrelevant for input), weak pull-up - Configures interrupt: - falling-edge trigger (`GPIO_FALLING_EDGE`) - clears and enables pin interrupt - enables the NVIC interrupt (`IntEnable(TACH_GPIO_INT)`) - Resets internal counters and state: - `g_tach_pulses`, `g_tach_rejects`, `g_last_edge_cycles` re-

---

`void GPIOFIntHandler(void)`

GPIO interrupt handler that counts tach pulses.

———
- Interrupt status is read and cleared first.
- For each falling edge on `TACH_GPIO_PIN`:
  - snapshots a timestamp via `timebase_cycles32()`
  - computes `delta = now - g_last_edge_cycles`
  - applies **minimum-edge-spacing reject**:
    - converts `TACH_MIN_EDGE_US` to cycles using `timebase_sysclk_hz()`
    - if `delta < min_cycles`: increments `g_tach_rejects` and ignores the

Glitch reject rationale:

- The project PWM is ~24.9kHz (period ~40µs). A `TACH_MIN_EDGE_US` default of **200µs** rejects most PWM-coupled "fake edges" on the tach line. - This is a diagnostic filter; it may need to change when we move to a period-based tach strategy like the ESP32 implementation.

`void tach_set_reporting(bool enabled)`

Enables/disables periodic reporting to UART0.

———— - When enabling: - schedules first report at `now + 500ms` - prints a one-time banner on UART0 with the active GPIO base/pin and configuration: - `TACHIN ON: gpio_base=0x... pin_mask=0x... edge=FALL pullup=WPU` - When disabling: - stops reporting - resets counters (`pulses`, `rejects`, `last_edge_cycles`) under

---

Important interaction note: - Reporting writes to **UART0 directly** (ROM `UARTCharPut`) and is **not** gated by `DEBUG ON/OFF`.

`bool tach_is_reporting(void)`

Returns whether periodic UART0 reporting is enabled.

`void tach_task(void)`

Periodic task (called from the main loop) that emits RPM diagnostics every 0.5s when enabled.
- Every 500ms:
- atomically snapshots and clears `g_tach_pulses` and `g_tach_rejects`
- computes an implied RPM using the current simplified model:

$\overline{\quad\quad}$
$RPM =$
$60 \cdot$
$pulses_{0.5s}$
This
comes
from:
- Win-
dow
$= 0.5s$
-
pulses/sec
$=$ `2 *`
`pulses_in_window`
- For a
2-
pulses-
per-
rev
fan:
`RPM =`
`(pulses/sec)`
`* 30`
`= 60`
*
`pulses_in_window`
-
Prints
one
line
on
UART0:
- `TACH`
`pulses=<n>`
`rejects=<n>`
`f=<Hz.t>Hz`
`rpm=<n>`

Note: after `TACHIN ON`, the firmware suppresses the **first 2 report windows** to avoid printing unstable/transient measurements during peripheral/pin switching.
Compile-time configuration knobs

These can be overridden at build time (e.g. via `-D...`):
- `TACH_GPIO_PERIPH`, `TACH_GPIO_BASE`, `TACH_GPIO_PIN`, `TACH_GPIO_INT`
- `TACH_MIN_EDGE_US` (default 200)

Known limitations (current diagnostic implementation)

- Counter-based windowing is sensitive to noise bursts; the `rejects` counter helps quantify that noise.
- Using a weak internal pull-up may be too susceptible on long wires / noisy grounds; external conditioning may be required.
- The current RPM conversion

- The current RPM conversion assumes 2 pulses/rev and a stable 0.5s window. Practical debug tip:
- Use `PSYN OFF` to force PF2 low and reduce PWM coupling while observing the tach signal and `rejects` behavior.

pwmin.c / pwmin.h

Interrupt-driven **PWM input sensing** (intended for the PSU's ~24.9kHz control PWM), plus optional periodic UART0 reporting.

Default pinning

- PWM input: **PF3 / GPIOF3**
- Intended peripheral function: **T1CCP1** (Timer1B capture)

Board note (TI TM4C1294 Connected LaunchPad, per `spmu365c.pdf`):

- **PF3 is not connected to a user LED**. User LEDs D3/D4 are on **PF4/PF0**.

Commands

- `PWMIN ON`: enables capture + prints one line every **1s** on UART0 (ICDI):
  - `PWMIN: f=<Hz>Hz duty=<pct.t>%` (duty shown with **0.1%** resolution)
- `PWMIN OFF`: stops the periodic printing
- `PWMINDBG ON`: enables verbose diagnostics to UART0 and prints a diagnostic dump
- `PWMINDBG OFF`: disables verbose diagnostics
- `PWMINDBG DUMP`: prints a one-time diagnostic dump (UART0)

Technique and rationale (Jan 2026)

This firmware originally implemented PWM sensing using **Timer1B capture** (PF3/T1CCP1). In practice, during bench testing we observed:

- PF3 was physically toggling at ~25kHz (confirmed by a short GPIO sampling probe).
- Timer1B capture produced **no capture events** (`raw=0 masked=0 edges_seen=0`).

To make PWM input sensing robust in this environment, `pwmin.c` now uses a **dual-path strategy**:

1. Attempt Timer1B capture (the "intended" peripheral path)
2. If capture events do not occur, fall back to **GPIOF PF3 both-edge interrupts** and compute timing in software

GPIO edge timestamp algorithm (fallback) The fallback path uses the system cycle timebase (`timebase_cycles32()`) as a timestamp source.

- Configure PF3 as GPIO input.
- Enable `GPIO_BOTH_EDGES` interrupt on PF3.
- In the GPIOF ISR hook (`pwmin_gpiof_isr()`):
  - Snapshot `now = timebase_cycles32()`.
  - Read the PF3 logic level.
  - If the level is HIGH (rising edge just occurred):
    - If a previous rise exists, compute `period_cycles = now - last_rise`.
    - Store `last_rise = now`.
  - Else (falling edge):
    - If a rise exists, compute `high_cycles = now - last_rise`.

Computed outputs:

- Normal display frequency (Hz): `f = sysclk_hz / period_cycles` (integer Hz)
- Normal display duty (0.1%): `duty_x10 = round(1000 * high_cycles / period_cycles)` then print as `duty_x10/10`

When verbose mode is enabled (`PWMINDBG ON`), an extra debug line is printed each second that includes **0.1 Hz** resolution:

- Debug display frequency (0.1 Hz): `f_x10 = round(10 * sysclk_hz / period_cycles)` then print as `f_x10/10`

This produces correct results for the project's ~24.9kHz PWM, and it avoids relying on a specific timer capture mux/capture behavior.

Caveats / things to avoid

- **Interrupt load**: both-edge interrupts at ~25kHz generate ~50k interrupts/sec. The ISR must be extremely small (no printing, no heavy math).
- **Latency/jitter**: any long critical sections or high-priority ISRs will add timestamp jitter. Duty is usually more sensitive than frequency.
- **Signal integrity**: fast edges + long wires can ring; use proper grounding and, if needed, series resistance / conditioning.
- **Voltage**: PF3 is **3.3V-only**. If the source can be pulled up to 5V, add level shifting/conditioning.
- **Assumptions**: this code assumes a stable `sysclk_hz` and a working `timebase_cycles32()`.

Diagnostic mode

The debug prints used to diagnose capture failures are preserved, but are **gated behind `PWMINDBG`** so normal `PWMIN` output stays clean.

With `PWMINDBG ON`, the 1 Hz reporting includes an additional line like:

- `PWMIN DBG: f=<Hz.t>Hz duty=<pct.t>% period_cycles=<n> high_cycles=<n> edges=<n>`

---

bothin.c / bothin.h

Coupled "lab friendly" reporting mode that prints **only** two measurements on UART0:

- PWM input duty with **0.1%** resolution
- Tach input RPM as **5 digits** (zero padded)

Command

- `BOTHIN ON`: enables BOTHIN output and ensures both measurement engines are running, while suppressing their individual periodic prints.
- `BOTHIN OFF`: stops BOTHIN output and restores the previous PWMIN/TACHIN reporting/printing state.

Output format (UART0)

- `BOTHIN: duty=<pct.t>% rpm=<00000..99999>`

Note: BOTHIN suppresses the first **2** 1-second reports after enabling (to avoid printing transient startup values).

---

tsyn.c / tsyn.h

Tach signal synthesis on PM3.

There are two distinct behaviors:

- **TSYN** (legacy): generates a bursty "tach-like" waveform on PM3 based on PSYN.
- **TACHSYN continuous**: generates a continuous square wave on PM3 at a specified frequency and duty.

PHASE commands (IBM PSU mimic)

UART3 adds fixed "phase" presets intended to mimic the IBM PSU boot/regime expectations:

- `PHASE1` / `PHASE 1`: PF2 PWM 24.9kHz @ 46% + PM3 TACHSYN 168Hz @ 50%
- `PHASE2` / `PHASE 2`: PF2 PWM 24.9kHz @ 54% + PM3 TACHSYN 235Hz @ 50%
- `PHASE1L` / `PHASE 1L`: PF2 PWM 24.9kHz @ 15% + PM3 TACHSYN 168Hz @ 50%
- `PHASE2L` / `PHASE 2L`: PF2 PWM 24.9kHz @ 21% + PM3 TACHSYN 235Hz @ 50%

TSYN BOOT commands (time-based TACHSYN profile)

UART3 commands:

- `TSYN BOOT BEGIN`: starts the boot-profile generator on PM3.
- `TSYN BOOT END`: stops the boot-profile generator.

Runtime behavior:

- The boot profile advances in the main loop (not inside an ISR) and is now advanced even while waiting for a UART3 DTR session, so it can run "immediately after reset" without requiring a UART3 connection.

TSYN COPY commands (TACHIN → TACHOUT mirroring)

UART3 commands:

- `TSYN COPY BEGIN`: mirrors PF1 (TACH IN) edge-for-edge onto PM3 (TACH OUT).
- `TSYN COPY END`: stops mirroring and restores the persisted `TACH DEFAULT` behavior (fallback: `PHASE1L`).

Implementation notes:

- While COPY is active, the PF1 interrupt is configured for **both edges**, and the ISR writes PM3 as a push-pull GPIO output to track the PF1 level.
- For tach diagnostics, the pulse counter still counts **falling edges** only.

Persistent default behavior (EEPROM)

UART3 commands:

- `TACH DEFAULT <1|2|1L|2L|BOOT|COPY>`: stores the selected default behavior in EEPROM.
- `TACH DEFAULT CURRENT`: prints the currently stored default.

Apply points:

- On boot, the firmware loads EEPROM and applies the stored default (if unset/invalid: `PHASE1L`).
- On `TSYN COPY END` and `TACH LOOPBACK END`, the firmware restores `TACH DEFAULT`.

Jan 2026 update: COPY + DEFAULT (quick reference)

What changed

- Added `TSYN COPY BEGIN/END`: mirrors PF1 (TACH IN) edge-for-edge onto PM3 (TACH OUT).
- Added EEPROM-backed `TACH DEFAULT <1|2|1L|2L|BOOT|COPY>` and `TACH DEFAULT CURRENT`.
- On boot, firmware loads EEPROM and applies the stored default (fallback: `PHASE1L`).
- On `TSYN COPY END` and `TACH LOOPBACK END`, firmware restores the stored `TACH DEFAULT`.

How to use

- Persist a default: `TACH DEFAULT 1L` (or `2`, `BOOT`, `COPY`, etc).
- Verify default: `TACH DEFAULT CURRENT`.
- Start mirroring: `TSYN COPY BEGIN`.
- Stop mirroring and restore default: `TSYN COPY END`.
- Check current mode states: `TSYN STATUS`.

Observed IBM PSU/server tach expectations (boot timeline)

The following describes lab observations of what the IBM server/PSU appears to expect on the fan tach feedback line during a cold AC-power boot.

All frequencies below are for a tach-like square wave at **~50% duty**, logic-level referenced to the PSU pull-up rail.

Observed sequence (timestamps approximate):

- **t = 0 to ~4.5s**: tach frequency transitions from ~60Hz down to ~50Hz.
- **~4.5s to ~7s**: tach holds near a stable ~50Hz.
- **~17s to ~28s**: tach ramps from ~50Hz up to the "phase 1" regime at ~168Hz.

Notes / caveats:

- The "~7s" and "~17s" markers were observed in the same session; treat the gap as "tach remains in the low regime until the ramp begins" unless/until refined with additional captures.
- Because the server may actively supervise tach plausibility, any experimentation that disconnects or forces tach low may trigger shutdown.

Phase transition behavior (steady-state expectations):

- **Phase 1 steady-state**: PSU commands ~46% PWM at 24.9kHz; fan tach feedback is ~168Hz at 50%.
- **Phase 2 steady-state**: PSU commands ~54% PWM at 24.9kHz; fan tach feedback is ~235–236Hz at 50%.

Planned firmware direction (experimental):

- Allow independent control of "real" PWM output applied to the fan versus the "fake" tach feedback supplied to the PSU.
- Start with fixed presets (`PHASE1/2/1L/2L`), then evolve toward a dynamic mapping (LUT/interpolation) from measured PSU PWM demand → synthesized TACHSYN frequency.

Electrical rationale: open-drain vs push-pull on PM3

- A **fan tach** is typically open-collector/open-drain, and the receiving electronics provides a pull-up.
- For direct connection to an unknown external pull-up rail, configuring PM3 as **open-drain** is the closest electrical match.

However, when using a small interface transistor (recommended for safety):

- Use a 2N3904 (or similar) as open-collector to the PSU tach input: emitter→GND, collector→PSU tach input.
- In that case, PM3 is driving the **base**, not the PSU node.
- For base drive, PM3 should be **push-pull** so the base sees a solid HIGH without needing an extra pull-up on the GPIO.

Firmware behavior:

- TSYN legacy output keeps PM3 in **open-drain** mode.
- PHASE commands set PM3 to **push-pull** drive for TACHSYN continuous mode (intended for 2N3904 base drive).

Measurement caveat: DMM averaging

- A DMM DC reading on a running tach line often reflects a time-average.
- For a 0↔3.3V, 50% duty waveform the average is ~1.65V.
- That reading alone does **not** prove the pull-up rail is 3.3V; use a scope (DC-coupled) and measure the actual Vhigh.

ESP32 reference guidance (for next tach algorithm iterations)

The ESP32 reference sketch (`fan_master_s2_final.ino`) uses a different strategy than the current TM4C implementation:

- ISR captures **period between edges** using `micros()` (stores `lastPeriod_us` and a `newTachMeasurement` flag).
- Main loop consumes that snapshot and computes:
  - `freq = 1e6 / period_us`
  - `RPM = (freq * 60) / PULSES_PER_REV`

This style is often more robust than "count pulses in a fixed window" when noise bursts are present, because you can qualify each edge-to-edge measurement and discard outliers without corrupting the whole window.

Concrete candidates to port into the TM4C path (still diagnostic/experimental until validated on the bench):

- **Hybrid measurement**: keep `pulses/rejects` window counters, but also track `last_period_us` (or `last_period_cycles`) from accepted edges.
- **Edge qualification**: require both a minimum edge spacing *and* a plausible period range (min/max RPM bounds) before accepting a new period.
- **Timeout-to-zero**: if no accepted edge arrives for a configurable time (e.g. 2–3 periods at min RPM), treat RPM as 0 or "stale".
- **Filtering**: apply a small moving average or median-of-3 over recent periods (or RPM) to reject sporadic glitches.
- **Expose tuning knobs**: make the key thresholds runtime-tunable via UART3 commands (e.g. `TACHMINUS <us>`, `TACHMAXRPM <n>`, `TACHTMO <ms>`), so lab work doesn't require rebuilds.

Notes observed in the ESP32 sketch worth mirroring during diagnostics:

- It uses a controlled "fake tach generator" with ~120µs low pulses to validate the algorithm path.
- It clamps commanded/target RPM to a safe min/max range (useful for sanity bounds during testing).

---

ui_uart3.c / ui_uart3.h

UI helpers for UART3 output discipline (banner/welcome/prompt).

`void ui_uart3_session_begin(void)`

Prints session-start UI:

- Deterministic ANSI "rainbow banner".
- A short welcome line.
- A single prompt.

Implementation constraint:

- Session-begin output must be deterministic and not rely on libc-heavy string searching/formatting to avoid stalls.

`void ui_uart3_puts(const char *s)`

Outputs a C string to UART3 via `UARTSend(..., UARTDEV_USER)`.

`void ui_uart3_prompt_once(void)`

Prints the prompt once (`ANSI_PROMPT + PROMPT_SYMBOL + ANSI_RESET`) and avoids duplicate prompt spam.

`void ui_uart3_prompt_force_next(void)`

Clears the "prompt already printed" latch so the next `ui_uart3_prompt_once()` will print.

---

diag_uart.c / diag_uart.h

Diagnostics helpers that write to UART0 (ICDI).

Important notes:

- These functions are intended for **non-ISR** contexts.
- The file contains both:
  - heap-based formatting helpers (`diag_vasprintf_heap`, `diag_snprintf_heap_send`) and
  - a minimal printf-like formatter (`diag_simple_sprintf`) plus global `sprintf/snprintf/printf` overrides.

UART0 output primitives

- `diag_putc(char c)`
- `diag_puts(const char *s)`
- `diag_put_hex32(uint32_t v)`
- `diag_put_u32_dec(uint32_t v)`
- `diag_put_ptr(const void *p)`

Heap formatting helpers

- `char *diag_vasprintf_heap(const char *fmt, va_list ap)`
- `char *diag_asprintf_heap(const char *fmt, ...)`
- `int diag_snprintf_heap_send(const char *fmt, ...)`

Memory/allocator diagnostics

- `void diag_sbrk_probe(void)`
- `void diag_test_malloc_with_gpio(void)`
- `void diag_test_malloc_sequence(void)`
- `void diag_print_memory_layout(void)`
- `void diag_print_sbrk_info(void)`
- `void diag_print_variable(const char *name, const void *addr, size_t size, size_t preview_limit)`
- `void diag_print_variables_summary(void)`

Memory protection helpers

- `void diag_check_memory_integrity(const char *context)`
- `void diag_check_stack_usage(const char *function_name)`
- `int diag_stack_bytes_used(void)`
- `int diag_heap_bytes_used(void)`

---

cmdline.c / cmdline.h (legacy)

This module provides an older UART3 command-line loop (`cmdline_run_until_disconnect`) and its own PSYN parsing.

Current status:

- The active firmware path in main.c uses `USERUARTIntHandler` + `commands_process_line()`.
- The build includes all `*.c` via the Makefile wildcard; however, link-time garbage collection (`--gc-sections`) typically discards this module unless referenced.

If you decide to use `cmdline_run_until_disconnect()` again:

- It expects a platform-visible `set_pwm_percent(uint32_t)` symbol (currently `set_pwm_percent` is `static` in main.c).

---

tools/uart_session.py (host-side)

Primary host automation tool for UART0/UART3 capture and scripted testing.

Key behaviors:

- Defaults: `--send-delay 0.6`, `--type-delay 0.02`.
- Preflight/postflight cleanup is enabled by default; can be disabled via `--no-preflight` / `--no-postflight`.
- For testing GOTCHA (real-time keystrokes), prefer `TYPE PPPPP` rather than a line-based `SEND` that appends ENTER.

Selecting UART device nodes (`/dev/ttyUSB0` vs `/dev/ttyUSB1`)

On Linux, the UART3 USB-serial adapter typically enumerates as `/dev/ttyUSB*`, but the index can change across hosts and reboots.

Recommended: use stable device names:

- List stable IDs: `ls -l /dev/serial/by-id/`
- Use those paths in host tooling:
  - `python3 tools/uart_session.py --uart0 /dev/ttyACM0 --uart3 /dev/serial/by-id/<adapter>`

Makefile override (quick and convenient):

- `make capture UART3_DEV=/dev/ttyUSB0`
- `make auto UART0_DEV=/dev/ttyACM0 UART3_DEV=/dev/ttyUSB1`

This is the intended way to "switch" without editing project files.

Hidden GOTCHA Feature (current spec)

- Trigger: **5 consecutive `P` keystrokes typed on UART3**.
- Immediate effects:
  - UART0 prints: `GOTCHA: PPPPP detected on UART3`.
  - PF4 LED flashes 5 times.
- Not a command; not listed in `HELP`.